

LeetCode Deep Dive

From First Instinct to Optimal Algorithm: Big-Integer Multiplication and Binary Search on the Answer

Week 7 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Outline

The process, recapped from Week 3

- 1 **First instinct.** The algorithm that follows most directly from re-reading the problem statement.
- 2 **Measure why it hurts.** Plug the problem's real constraints into the complexity and watch it blow up.
- 3 **Find the structural insight.** The property of the problem that lets you avoid redoing work.
- 4 **Implement the optimal algorithm,** trace it by hand, then look for the *programming-level* tricks that make the implementation itself correct and fast.

This week

Both problems below are, at heart, **this week's** lecture ideas wearing a LeetCode costume: grade-school multiplication (Section 2 of the notes) and “two representations, pick the cheap one to search in” (the same divide-and-conquer instinct that drives the FFT).

LeetCode 43 – Multiply Strings (Medium)

Given two non-negative integers `num1` and `num2` represented as strings, return the product, also as a string. You may **not** convert the inputs to a built-in big-integer type and multiply directly.

- Constraints (LeetCode): $1 \leq |\text{num1}|, |\text{num2}| \leq 200$, both contain only digits, no leading zeros unless the number itself is "0".
- This is exactly this week's "grade-school recurrence" (Section 2 of the lecture notes), with the "no built-in bigint" rule forcing you to actually implement it.

Worked example

`num1 = "123", num2 = "456".`

- By hand: $123 \times 456 = 56,088$.
- The grade-school algorithm builds this digit by digit, exactly the way you were taught in primary school: multiply 123 by each digit of 456 (ones, tens, hundreds), shift, and add.
- We will trace the array-based version of this below and confirm it produces "56088".

First instinct: multiplication as repeated addition

Most direct reading of “multiply”

Multiplication is repeated addition: to compute $a \times b$, add a to itself, b times.

```
1: function MULTIPLY( $a, b$ )  
2:    $result \leftarrow 0$   
3:   for  $k \leftarrow 1$  to  $b$  do  
4:      $result \leftarrow \text{STRINGADD}(result, a)$   
5:   end for  
6: end function
```

▷ $b =$ the *value* of num2

Each STRINGADD is cheap ($O(n)$ on n -digit numbers) – but you repeat it b times, where b is the *value* of num2, not its *length*.

Why this is bad

- num2 can have up to 200 digits, i.e. a value up to $10^{200} - 1$.
- Repeated addition costs $O(b \cdot n)$ where $b \approx 10^{200}$ – utterly infeasible; even one operation per nanosecond would need vastly longer than the age of the universe.
- The bug in the instinct: it treats the input's *numeric value* as the size parameter, when the real size parameter is the *number of digits*. This is the same trap as confusing n with 2^n .

The smell to notice

Whenever “how many times do I repeat this” is tied to a *value* rather than an *input length*, the algorithm is almost certainly exponential in the actual input size (the number of digits).

The structural insight

- Grade-school long multiplication never repeats work proportional to the *value* – it repeats work proportional to the number of *digit pairs*, exactly $|\text{num1}| \times |\text{num2}|$.
- Every digit pair $(\text{num1}[i], \text{num2}[j])$ contributes to *exactly one* pair of positions in the result: a units position $i + j + 1$ and a carry position $i + j$ (using 0-indexing from the left of each string).
- So the whole algorithm is: allocate a result array of size $|\text{num1}| + |\text{num2}|$, and for every digit pair, add its contribution directly into those two positions – no repetition tied to the numbers' *values* at all.

Optimal algorithm: pseudocode

```
1: function MULTIPLY(num1, num2)
2:   if num1 = "0" or num2 = "0" then return "0"
3:   end if
4:    $n, m \leftarrow |\text{num1}|, |\text{num2}|$ 
5:   result  $\leftarrow$  array of  $n + m$  zeros
6:   for  $i \leftarrow n - 1$  downto 0 do
7:     for  $j \leftarrow m - 1$  downto 0 do
8:        $mul \leftarrow \text{num1}[i] \times \text{num2}[j]$ 
9:        $p_1, p_2 \leftarrow i + j, i + j + 1$ 
10:       $total \leftarrow mul + \text{result}[p_2]$ 
11:       $\text{result}[p_2] \leftarrow total \bmod 10$ 
12:       $\text{result}[p_1] \leftarrow \text{result}[p_1] + \lfloor total / 10 \rfloor$ 
13:    end for
14:  end for
15:  return result joined to a string, with leading zeros stripped
16: end function
```

▷ as digits

$O(nm)$ total – exactly this week's $\Theta(n^2)$ grade-school bound (here $n \neq m$ in general, so $O(nm)$).

Trace on the worked example

num1 = "123" ($i = 0, 1, 2 \rightarrow$ digits 1, 2, 3), num2 = "456" ($j = 0, 1, 2 \rightarrow$ digits 4, 5, 6). Result array has indices 0..5, all start at 0.

after processing	result array (index 0 $\rightarrow$ 5)
$i = 2$ (digit 3) against all of num2	[0, 0, 1, 3, 6, 8]
$i = 1$ (digit 2) against all of num2	[0, 1, 0, 4, 8, 8]
$i = 0$ (digit 1) against all of num2	[0, 5, 6, 0, 8, 8]

Strip the leading zero: "56088" – matches $123 \times 456 = 56,088$ exactly. Note position 0 never overflows past a single digit: it is written *once*, by the single pair $i=j=0$, receiving at most one carry.

Complexity: naive vs. optimal

Approach	Time	At $ \text{num2} = 200$
Repeated addition	$O(b \cdot n)$, $b \approx 10^{200}$	astronomically infeasible
Grade-school digit array	$O(nm)$	$\leq 200 \times 200 = 40,000$ ops

Why not Karatsuba or the FFT here?

This week's faster methods (Karatsuba $\Theta(n^{1.585})$, FFT-based $O(n \log n)$) exist precisely for *this* problem in general – but at $n, m \leq 200$, $O(nm) = 40,000$ is already instant, and the constant-factor overhead of Karatsuba/FFT would make the code slower in practice. Matching the algorithm to the actual input size, not just the asymptotic label, is itself an engineering skill.

Python implementation

```
1 def multiply(num1: str, num2: str) -> str:
2     if num1 == "0" or num2 == "0":
3         return "0"
4     n, m = len(num1), len(num2)
5     result = [0] * (n + m)
6     for i in range(n - 1, -1, -1):
7         d1 = ord(num1[i]) - ord('0')
8         for j in range(m - 1, -1, -1):
9             d2 = ord(num2[j]) - ord('0')
10            mul = d1 * d2
11            p1, p2 = i + j, i + j + 1
12            total = mul + result[p2]
13            result[p2] = total % 10
14            result[p1] += total // 10
15
16     idx = 0
17     while idx < len(result) - 1 and result[idx] == 0:
18         idx += 1
19     return ''.join(map(str, result[idx:]))
```

Implementation tips & tricks (the programming side)

- **Never build the result with string concatenation in the inner loop.** Python strings are immutable, so `s = s + digit` inside an $O(nm)$ double loop would silently turn the algorithm into $O(nm \cdot (n + m))$. Accumulate into a mutable list of integers (`result`) and `join` exactly once at the end.
- `result[p1] += total // 10`, **not** `=`. Using `+=` lets a position accumulate carries from *multiple* digit pairs across different (i, j) before it is finally read; overwriting it would silently drop a carry.
- **Handle "0" as a special case up front.** Without it, multiplying by "0" still produces a technically-correct all-zero array, but you would then need extra logic to avoid returning a string of multiple zeros.

Problem statement

LeetCode 4 – Median of Two Sorted Arrays (Hard)

Given two sorted arrays `nums1` (size m) and `nums2` (size n), return the median of the combined sorted array, in $O(\log(m + n))$ time.

- Constraints (LeetCode): $0 \leq m, n \leq 1000$, $1 \leq m + n \leq 2000$.
- Merging the two arrays is only $O(m + n)$ – fast enough in practice. The point of demanding $O(\log(m + n))$ is to force the “two representations, search in the cheap one” idea this week’s FFT section teaches, applied to *position* instead of *value*.

Worked example

`nums1 = [1, 3, 8]` ($m = 3$), `nums2 = [7, 9, 10, 11]` ($n = 4$).

- Merged: `[1, 3, 7, 8, 9, 10, 11]` – 7 elements (odd), so the median is the **4th smallest**: 8.
- We will find this *without merging*, by binary-searching for a “partition” of both arrays directly.

First instinct: merge, then index

Most direct reading of “median of the combined array”

Merge `nums1` and `nums2` (the merge step from last week's mergesort), then read off the middle element(s).

- 1: **function** MEDIANNAIVE(`nums1`, `nums2`)
- 2: `merged` $\leftarrow$ merge `nums1` and `nums2` (two-pointer, $O(m + n)$)
- 3: **return** the middle element (or average of the two middle elements)
- 4: **end function**

This is *correct* and even reasonably fast – but it is $\Theta(m + n)$, not $O(\log(m + n))$, and it materializes an entire merged array just to read one or two values from it.

Why this is bad (for the bound the problem actually asks for)

- At $m + n = 2000$, $\Theta(m + n)$ is 2000 steps – utterly fine in practice. So why does the problem insist on $O(\log(m + n)) \approx 11$ steps?
- Because the *technique* matters more than these particular numbers: in a setting where `nums1` and `nums2` are two huge sorted shards (e.g. on different machines, or too large to ever fully scan), $O(m + n)$ means reading *everything*, while $O(\log(m + n))$ means a handful of random-access probes.

The smell to notice

“Merge everything, then look at one position” throws away almost all of the work you just did. If you only need *one* answer, ask whether you can binary search directly for *it*, the same way this week’s FFT searches for the right representation instead of computing every intermediate value.

The structural insight

- The median splits the combined array into a left half and a right half of (almost) equal size, where **every** element on the left is $\leq$ **every** element on the right.
- That split is really a choice of *how many elements come from each array*: if i elements of `nums1` are on the left, exactly *half* $- i$ elements of `nums2` must be too, where $half = \lfloor (m + n + 1)/2 \rfloor$.
- As i increases, `nums1`'s left-boundary value only increases and `nums2`'s left-boundary value only decreases – **monotonic**, so binary search on $i \in [0, m]$ finds the unique correct split in $O(\log m)$ (search the *shorter* array, so this is $O(\log \min(m, n))$).

Optimal algorithm: pseudocode

```
1: function FINDMEDIAN(A, B)
2:    $m, n \leftarrow |A|, |B|$ ;  $half \leftarrow \lfloor (m + n + 1)/2 \rfloor$ 
3:    $lo, hi \leftarrow 0, m$ 
4:   while  $lo \leq hi$  do
5:      $i \leftarrow \lfloor (lo + hi)/2 \rfloor$ ;  $j \leftarrow half - i$ 
6:      $A_L \leftarrow A[i - 1]$  if  $i > 0$  else  $-\infty$ ;  $A_R \leftarrow A[i]$  if  $i < m$  else  $+\infty$ 
7:      $B_L \leftarrow B[j - 1]$  if  $j > 0$  else  $-\infty$ ;  $B_R \leftarrow B[j]$  if  $j < n$  else  $+\infty$ 
8:     if  $A_L \leq B_R$  and  $B_L \leq A_R$  then
9:       if  $(m + n)$  is odd then return  $\max(A_L, B_L)$ 
10:      else return  $(\max(A_L, B_L) + \min(A_R, B_R))/2$ 
11:    end if
12:    else if  $A_L > B_R$  then  $hi \leftarrow i - 1$ 
13:    else  $lo \leftarrow i + 1$ 
14:    end if
15:  end while
16: end function
```

▷ assume $|A| \leq |B|$ (swap first otherwise)

Trace on the worked example

$A = [1, 3, 8]$ ($m = 3$), $B = [7, 9, 10, 11]$ ($n = 4$), $half = \lceil 7/2 \rceil = 4$. Binary search $i \in [0, 3]$.

step	i	j	A_L	A_R	B_L	B_R	verdict
1	1	3	1	3	10	11	$B_L > A_R$ ($10 > 3$): need larger i
2	2	2	3	8	9	10	$B_L > A_R$ ($9 > 8$): need larger i
3	3	1	8	$+\infty$	7	9	both hold: done

$m + n = 7$ is odd, so median = $\max(A_L, B_L) = \max(8, 7) = \boxed{8}$ – matches the merged array's 4th-smallest element, found in only 3 probes instead of merging all 7.

Complexity: naive vs. optimal

Approach	Time	At $m = n = 1000$
Sort the concatenation	$O((m+n) \log(m+n))$	$\sim 2000 \times 11 \approx 22,000$
Merge, then index	$O(m + n)$	2000 steps
Binary search on the partition	$O(\log \min(m, n))$	≤ 10 steps

Python implementation

```
1 def findMedianSortedArrays(nums1, nums2):
2     A, B = (nums1, nums2) if len(nums1) <= len(nums2) else (nums2, nums1)
3     m, n = len(A), len(B)
4     half = (m + n + 1) // 2
5     lo, hi = 0, m
6     while lo <= hi:
7         i = (lo + hi) // 2
8         j = half - i
9         a_left = A[i - 1] if i > 0 else float('-inf')
10        a_right = A[i] if i < m else float('inf')
11        b_left = B[j - 1] if j > 0 else float('-inf')
12        b_right = B[j] if j < n else float('inf')
13        if a_left <= b_right and b_left <= a_right:
14            if (m + n) % 2:
15                return max(a_left, b_left)
16            return (max(a_left, b_left) + min(a_right, b_right)) / 2
17        elif a_left > b_right:
18            hi = i - 1
19        else:
20            lo = i + 1
```

Implementation tips & tricks (the programming side)

- **Always binary search on the shorter array.** Searching on the longer one still terminates correctly, but the bound becomes $O(\log \max(m, n))$ instead of $O(\log \min(m, n))$ – a real, measurable difference when the two inputs are very different sizes.
- **Use $\pm\infty$ sentinels for out-of-range partitions** rather than scattering `if i == 0` / `if i == m` special cases through the comparison logic – it collapses four edge cases into the same two comparisons used in the general case.
- **Binary search bounds are $[0, m]$, not $[0, m-1]$.** There are $m + 1$ possible ways to split A (from “take none of A ” to “take all of A ”), so `hi` must start at m – an easy off-by-one to get wrong.
- **Python's `/` already returns a float**, so the even-length average needs no explicit cast – unlike languages with C-style integer division, where forgetting to cast before dividing silently truncates the result.

Summary: two problems, one lesson

- 1 **Multiply Strings** is this week's grade-school $\Theta(nm)$ multiplication, forced into code – the naive trap was confusing a number's *value* with its *length* as the real input size.
- 2 **Median of Two Sorted Arrays** reuses this week's “two representations, search the cheap one” idea (the same instinct behind evaluating a polynomial at roots of unity): instead of materializing the merged array, binary search directly for the one partition you actually need.
- 3 Both times, the naive instinct was *correct but did unnecessary work* – repeating additions tied to a value, or building a whole merged array to read one entry. The fix was the same shape: find the invariant (digit-pair independence; partition monotonicity) that lets you compute only what the answer needs.
- 4 The implementation tricks (accumulate-then-join, $\pm\infty$ sentinels, binary-search-the-shorter-array) are a second, independent layer on top of an already-optimal algorithm.

Keep practicing

LEETCODE.md lists six more Week 7 problems (*Search a 2D Matrix II*, *Kth Smallest Element in a Sorted Matrix*, *The Skyline Problem*, *Count of Range Sum*, *Find K-th Smallest Pair Distance*, *Super Pow*) – try the same four-step process on each.